○ E-LEARNING · STUDY NOTES

Nicloud · AWS Security

Module 1 – Service Control Policies in Practice

Contents

Learning objectives 2
Architecture 2
Service Control Policies (SCP) 3
Why Deny instead of Allow 3
Restricting Regions 4
Example: Regions in this repo 4
Locking down IAM and the security baseline 4
Required encryption 5
Example: the IfExists trap 5
Required tags 6
Requiring IMDSv2 6
Example: IMDSv2 in this repo 6
Country-based block 7
Example: the exceptions that keep AWS working 7
SCP size limit 7
Testing an SCP before attaching it 8
Summary 8
Quiz 8
Additional resources 9

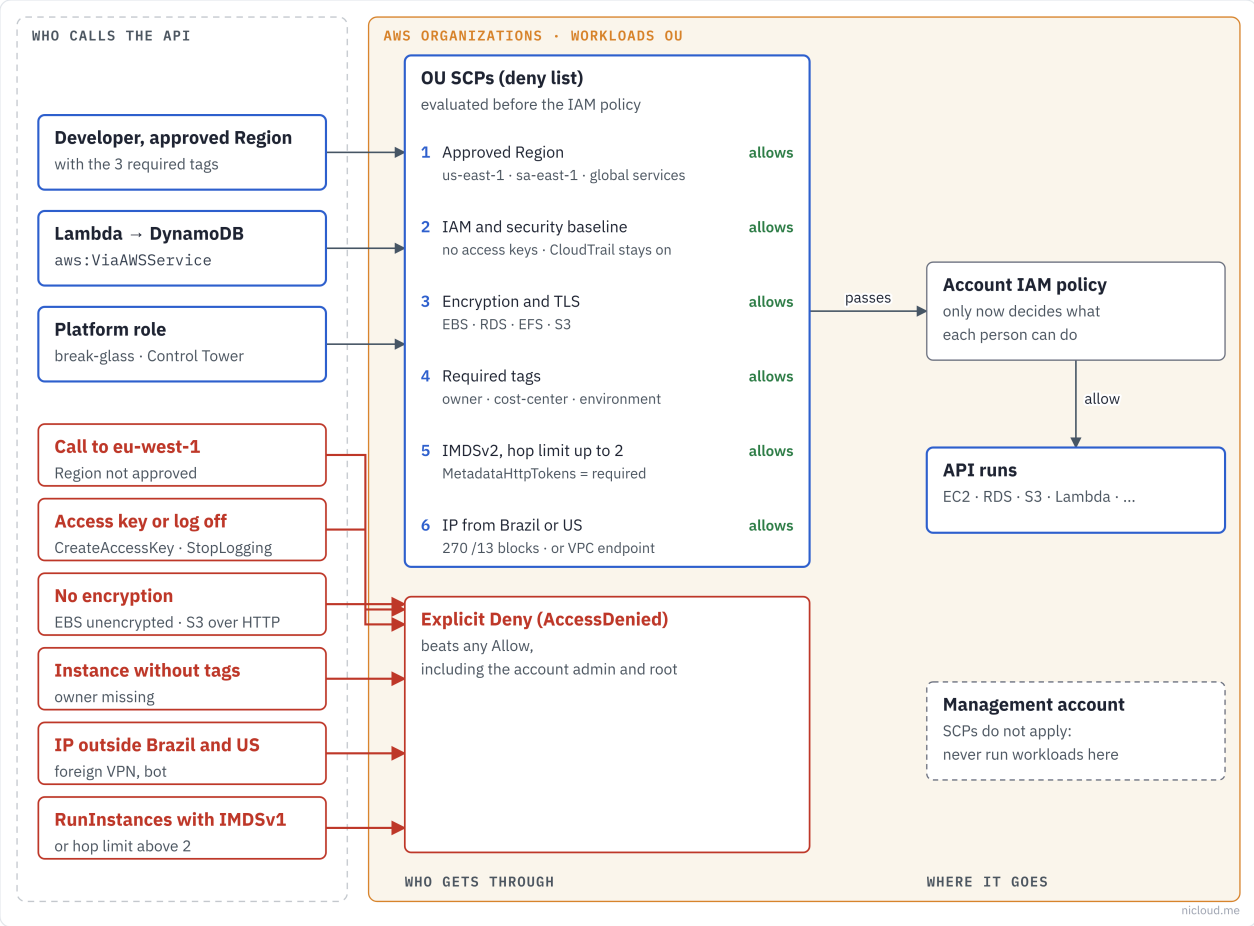
Learning objectives

In this module, you will learn how to:

- Describe what a **Service Control Policy (SCP)** does and what it does not do.
- Differentiate an SCP from an IAM policy.
- Explain why the SCPs in this repository use **Deny** instead of Allow.
- Read SCP conditions (`StringNotEquals`, `NumericGreaterThan`, `NotIpAddress`, `BoolIfExists`).
- Restrict **Regions** without breaking global services.
- Lock down **IAM** and the **security baseline** (CloudTrail, Config, GuardDuty) against people with admin access.
- Require **encryption** on S3, EBS, RDS and EFS, and avoid the `IfExists` trap.
- Require owner, cost center and environment **tags** when resources are created.
- Require **IMDSv2** on every EC2 instance in the organization.
- Build a country-based block without breaking AWS service-to-service calls.
- Respect the SCP size limit and check it when it changes.
- **Test** an SCP before attaching it, with IAM Access Analyzer and automated tests.

Architecture

Overview of what this repository does, in the Nicloud diagram style: callers on the left, the six rules in the middle, destinations on the right. Blue gets through; red hits the Deny.



Service Control Policies (SCP)

A **Service Control Policy** is an **AWS Organizations** rule that sets the **maximum permissions** the accounts in an organizational unit (OU) can have. It never grants access: it only takes it away.

Think of the SCP as the ceiling of a building. IAM policies decide what each person can do inside, but nobody goes through the ceiling, not even the root user of a member account.

- An action is allowed only if **both the SCP and the IAM policy allow it**.
- SCPs do not apply to the organization's **management account**.
- SCPs do not apply to **service-linked roles**.

When to use

Use SCPs for rules that **no** account may break, by mistake or with admin access: allowed regions, mandatory encryption, instance security standards, source restrictions.

Why Deny instead of Allow

This repository follows a **deny list** strategy: the default `FullAWSAccess` SCP stays attached and each file here adds one specific **Deny**. An explicit Deny always wins over any Allow.

The alternative (an allow list, which removes `FullAWSAccess` and lists only what is permitted) gives more control, but every new service requires an SCP change. For a reusable library, the deny list is safer to

roll out without blocking anyone.

Restricting Regions

A **Region restriction** denies every action outside the approved Regions. It prevents resources in places nobody monitors, with no budget and outside the compliance scope.

The catch is **global services**: IAM, STS, Organizations, Route 53, CloudFront, Support and billing have their endpoint in `us-east-1`. If the SCP denies everything outside `sa-east-1`, the console stops working and nobody can assume a role.

Example: Regions in this repo

```
// policies/region-restriction/deny-non-approved-regions.json
{
  "Sid": "DenyOutsideApprovedRegions",
  "Effect": "Deny",
  "NotAction": ["iam:*", "sts:*", "route53:*", "cloudfront:*", "support:*",
"..."],
  "Resource": "*",
  "Condition": {
    "StringNotEquals": { "aws:RequestedRegion": ["us-east-1", "sa-east-1"] },
    "ArnNotLike": { "aws:PrincipalArn":
["arn:aws:iam::*:role/OrganizationAccountAccessRole", "..."] }
  }
}
```

How it works

(1) `NotAction` instead of `Action`

The Deny applies to **every action except** the listed ones, the global services from the AWS example. A new global service has to be added.

(2) Platform roles are exempt

`ArnNotLike` on `aws:PrincipalArn` exempts the recovery roles (`OrganizationAccountAccessRole`, `AWSControlTowerExecution` and `break-glass-admin`), so the platform team can always get in and fix things.

Locking down IAM and the security baseline

The `iam-lockdown` policy groups five rules an attacker would try to break first:

Statement	Denies	Why
`DenyLongLivedIamUserCredentials`	creating IAM users, access keys and console passwords	people sign in through SSO; workloads use roles
`ProtectPlatformRoles`	changing the recovery roles and the SSO roles	a compromised developer role cannot take over the account
`ProtectSecurityBaseline`	turning off CloudTrail, Config, GuardDuty, Security Hub and Access Analyzer	the first thing an attacker does is erase the trail
`DenyLeavingOrganization`	leaving the organization	outside the organization, no SCP applies
`DenyRootUser`	any action by the root user of member accounts	root only for what requires root

Note that `DenyLeavingOrganization` has no exception, not even for the admin role.

Required encryption

The `require-encryption` policy denies S3 without TLS (`aws:SecureTransport = false`) and EBS volumes, RDS databases and EFS file systems created without encryption. It also blocks turning off EBS default encryption.

Example: the IfExists trap

The first version of the S3 upload rule used `StringNotEqualsIfExists`. The automated test caught the bug: **an `...IfExists` operator is true when the key is missing**. Most uploads do not send the encryption header (S3 encrypts every new object by default since 2023), so the rule would deny almost everything.

```
// policies/require-encryption/require-encryption.json
"Condition": {
  "Null":            { "s3:x-amz-server-side-encryption": "false" },
  "StringNotEquals": { "s3:x-amz-server-side-encryption": ["AES256", "aws:kms",
"aws:kms:dsse"] }
}
```

`Null = false` means "the key **exists**". Now the Deny only applies when the upload asks for an encryption type outside the list.

Lesson

Negated operators (`StringNotEquals`, `ArnNotLike`, `NotIpAddress`) and those ending in `IfExists` are **true when the key is missing**. Always ask: what happens when the key is not in the request?

Required tags

The `tag-enforcement` policy requires `owner`, `cost-center` and `environment` (`dev`, `staging` or `prod`) when creating EC2 instances, EBS volumes, RDS databases and Lambda functions. The condition is `Null` on `aws:RequestTag/<key>`: if the tag is not in the request, it is denied.

`Resource` limits the rule to those types. `RunInstances` also touches subnets, security groups and network interfaces; without that filter the whole launch would fail. A fourth rule prevents deleting these tags later.

Requiring IMDSv2

The **Instance Metadata Service (IMDS)** lives at `169.254.169.254` and lets an instance read data about itself, including the **temporary credentials of its role**. Version 1 answers a plain GET, so an **SSRF** attack (tricking the application into making a request) can steal those credentials. **Version 2** requires a token obtained first with a PUT.

Example: IMDSv2 in this repo

```
// policies/imdsv2-enforcement/enforce-imdsv2.json
{
  "Sid": "RequireIMDSv2OnRunInstances",
  "Effect": "Deny",
  "Action": "ec2:RunInstances",
  "Resource": "arn:aws:ec2:*:*:instance/*",
  "Condition": {
    "StringNotEquals": { "ec2:MetadataHttpTokens": "required" }
  }
}
```

How it works

(1) Require the token

The first statement denies `RunInstances` when `MetadataHttpTokens` is not `required`. New instances can only launch with IMDSv2.

(2) Limit the hops

The second statement denies when `MetadataHttpPutResponseHopLimit` is greater than **2**. The hop limit is how many network hops the token response survives. With 2, a container on the instance can still reach IMDS; with more, it gets easier for the token to leak out.

What it does not cover

This SCP applies to **new** instances. Existing IMDSv1 instances stay as they are: migrate them with `modify-instance-metadata-options` and, to stop anyone from switching back, also deny that action when the value is not `required`.

Country-based block

The second policy denies API calls coming from IPs outside Brazil and the United States. The main condition is **`NotIpAddress`** on `aws:SourceIp`: if the source IP is **not** on the list, the call is denied.

The catch is that not every call carries a person's IP. When Lambda calls DynamoDB, or ECS pulls an image from ECR, the caller is AWS itself.

Example: the exceptions that keep AWS working

```
// policies/geo-restriction/deny-access-from-non-allowed-countries.json
"Condition": {
  "NotIpAddress":   { "aws:SourceIp": ["177.0.0.0/13", "..."] },
  "BoolIfExists":   { "aws:ViaAWSService": "false" },
  "StringNotLike":  { "aws:SourceVpce": "vpce-*" }
}
```

All three conditions must be true for the Deny to apply:

Condition	What it guarantees
`NotIpAddress`	the IP is outside the allowed list
`BoolIfExists` `aws:ViaAWSService: false`	it is not an AWS service calling on someone's behalf
`StringNotLike` `aws:SourceVpce: vpce-*`	it did not come through a VPC Endpoint (which has no public IP)

Where the IPs come from

`generate-country-cidrs.sh` downloads the delegation files from the regional registries (**LACNIC** for Brazil, **ARIN** for the United States), filters by country and aggregates the blocks into **/13**.

SCP size limit

Today an SCP accepts up to **10,240 characters** (the 5,120 limit applies to **Resource Control Policies**). In the console, whitespace between JSON elements is removed and not counted; through the CLI or SDK the text is saved as is and everything counts.

The country-block file is 8,375 characters formatted and **4,691 minified**. Both fit today. The **/13** blocks were chosen when the limit was 5,120: with the current limit, smaller and more precise blocks are possible, letting in fewer IPs from neighboring countries.

Lesson

Service limits change. Before making a trade-off because of a limit, check the number in today's documentation and write the date in the README.

Testing an SCP before attaching it

A wrong SCP takes down all of production at once. This repository uses two layers of tests:

1. **IAM Access Analyzer** (`validate-policy` with `--policy-type SERVICE_CONTROL_POLICY`) checks the syntax and flags bad practices. It caught a `StringLike` on `aws:PrincipalArn` that should be `ArnLike`.
2. **Automated tests** (`pytest`): a small SCP evaluator (`tests/scp_eval.py`) simulates 20 scenarios, such as "an instance without the `environment` tag is denied" and "IAM keeps working outside the approved Regions". CI runs them on every push.

Even with green tests, attach one policy at a time, first to a sandbox OU. Then watch CloudTrail for `AccessDenied` with "explicit deny in a service control policy".

Summary

An **SCP** is the permission ceiling for the accounts in an organization: it only takes away, never grants, and it binds even the member account's root user. This repository uses a **deny list** strategy, keeping `FullAWSAccess` and adding specific denies.

The **IMDSv2** policy prevents instances from launching with version 1 of the metadata service, the most common path to credential theft through **SSRF**, and caps the **hop limit** at 2.

The **country-block** policy denies calls from outside Brazil and the United States, but only after excluding calls from **AWS services** and **VPC Endpoints**. Without those two exceptions, it would take down your own infrastructure.

The **Region restriction** denies everything outside the approved Regions except the **global services** in `NotAction`. **iam-lockdown** forbids IAM users and access keys, protects the recovery roles, blocks turning off CloudTrail, Config and GuardDuty, blocks leaving the organization and blocks root. **Required encryption** enforces TLS on S3 and encrypted disks and databases. **Required tags** enforce owner, cost center and environment at creation.

Watch out for negated and `IfExists` operators: they are true when the key is missing.

And every SCP must fit in **10,240 characters**: minify the JSON when sending it through the CLI, because whitespace counts there.

Quiz

An admin in a member account has `AdministratorAccess` but cannot launch an instance. The IAM policy is correct. What should you check?

- An SCP on the account's OU. SCPs limit even admins and the member account's root user.

You must ensure no new instance in the organization uses IMDSv1. Which condition goes in the Deny on `ec2:RunInstances`?

- `StringNotEquals` on `ec2:MetadataHttpTokens` with the value `required`.

After applying the country block, Lambda stopped writing to DynamoDB. Which exception is probably missing?

- `aws:ViaAWSService`. The call comes from the service, not from a person's IP.

An SCP with 11,000 formatted characters fails when saved through the CLI, but the minified content has 9,000. What do you do?

- Minify the JSON before sending it. Whitespace counts in the CLI; the SCP limit is 10,240 characters.

Does the IMDSv2 SCP fix existing IMDSv1 instances?

- No. It only applies to `RunInstances`. Existing instances must be migrated with `modify-instance-metadata-options`.

A developer cannot create a role while calling `eu-west-1`, and the Region SCP only allows `us-east-1` and `sa-east-1`. Is that expected?

- No. IAM is a global service listed in `NotAction`, so creating roles works from any Region. If it was denied, check that the global services list is complete.

A rule with `StringNotEqualsIfExists` on `s3:x-amz-server-side-encryption` started denying almost every upload. Why?

- `IfExists` is true when the key is missing, and most uploads do not send that header. Use `Null = false` together with `StringNotEquals`.

Why does tag-enforcement limit `Resource` to `instance/*` and `volume/*` on `RunInstances`?

- Because `RunInstances` also touches subnets, security groups and network interfaces, which do not get the tags. Without the filter, every launch would be denied.

Which iam-lockdown statement has no exception, not even for the admin role?

- `DenyLeavingOrganization`. An account outside the organization loses every SCP.

Do SCPs apply to the organization's management account?

- No. That is why nothing should run in the management account besides administering the organization.

Additional resources

- Service control policies (AWS Organizations)
- SCP examples
- AWS Organizations quotas
- Configure the instance metadata service (IMDS)
- Policy validation with IAM Access Analyzer
- IAM condition operators
- IAM global condition keys